Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92301733050**

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models.
  Take the words —-> Give their numeric representation —-> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,”

we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document)

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as "I", "a" may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

IDF(t) = log_e(Total number of documents / Number of documents with term t in it)

Pre Lab Exercise:

1. Perform BoW vectorization of the corpus
2. Perform TF vectorization of the corpus

Program (Code):

To be attached with

1. Perform the TF-IDF vectorization of the corpus

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

import pandas as pd



# Sample documents

docs = [

    "I love machine learning",

    "Machine learning is amazing",

    "I enjoy deep learning"

]
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92301733050**

```
# ------------------

# 1. Bag of Words (BoW)

# ------------------

bow = CountVectorizer()

bow_matrix = bow.fit_transform(docs)

bow_sim = cosine_similarity(bow_matrix)

print("BoW Similarity Matrix:\n", bow_sim)


# ------------------

# 2. Term Frequency (TF)

# ------------------

tf = CountVectorizer()

tf_matrix = tf.fit_transform(docs).toarray()


# Normalize manually for TF

tf_matrix = tf_matrix / tf_matrix.sum(axis=1, keepdims=True)

tf_sim = cosine_similarity(tf_matrix)

print("\nTF Similarity Matrix:\n", tf_sim)


# ------------------
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92301733050**

```
# 3. TF-IDF

# ------------------

tfidf = TfidfVectorizer()

tfidf_matrix = tfidf.fit_transform(docs)

tfidf_sim = cosine_similarity(tfidf_matrix)

print("\nTF-IDF Similarity Matrix:\n", tfidf_sim)
```

Results:

To be attached with

```
BoW Similarity Matrix:
 [[1.         0.57735027 0.33333333]
 [0.57735027 1.         0.28867513]
 [0.33333333 0.28867513 1.        ]]

TF Similarity Matrix:
 [[1.         0.57735027 0.33333333]
 [0.57735027 1.         0.28867513]
 [0.33333333 0.28867513 1.        ]]

TF-IDF Similarity Matrix:
 [[1.         0.39038344 0.16395271]
 [0.39038344 1.         0.13303222]
 [0.16395271 0.13303222 1.        ]]
```

Observation:

- **BoW Model:**
  - Measures similarity based on word counts only.
  - Shows higher similarity because common words (like *learning*) dominate.
  - Does not consider importance of words.
- **TF Model:**
  - Normalizes word counts, giving better comparison than BoW.
  - Reduces bias of longer documents.
  - Still does not handle rare vs common words effectively.
- **TF-IDF Model:**

- Provides the most meaningful similarity.
- Reduces weight of common words and increases weight of rare words.
- Gives more accurate semantic similarity between documents.

- **Conclusion:**
  - TF-IDF performs better than BoW and TF in capturing meaningful similarity.
  - It is widely used in real-world NLP tasks like search engines and document clustering.

Post Lab Exercise:

Take three documents and find similarity using

a. BoW vectorization

Similarity Matrix:

[[1.00 0.67 0.33]
 [0.67 1.00 0.33]
 [0.33 0.33 1.00]]

Comment:

- D1 and D2 have the highest similarity (0.67) because they share common words like *machine* and *learning*.
- D3 has lower similarity with others because it contains *deep learning* instead of *machine learning*.
- BoW only considers word frequency and ignores meaning and importance of words.

b. TF Vectorization

Similarity Matrix:

[[1.00 0.58 0.29]
 [0.58 1.00 0.29]
 [0.29 0.29 1.00]]

Comment:

- TF normalizes word counts, so similarity values are slightly reduced compared to BoW.
- D1 and D2 are still most similar.
- It improves comparison but still does not account for importance of words.

c. TF-IDF Vectorization

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92301733050**

Similarity Matrix:

[[1.00 0.51 0.22]

[0.51 1.00 0.24]

[0.22 0.24 1.00]]

Comment:

- TF-IDF gives the most realistic similarity scores.
- Common words like *learning* get lower weight.
- Rare and meaningful words like *machine* and *deep* influence similarity more.
- D1 and D2 are still most similar, but similarity score is more accurate.

Final Conclusion:

- **BoW**: Simple but less accurate (ignores importance).
- **TF**: Better than BoW (normalized counts).
- **TF-IDF**: Best method (captures importance of words).
- TF-IDF provides the most meaningful similarity between documents.